

Journée de l'écoconception numérique 2027

Green Craft

10 actions concrètes pour un numérique soutenable que tu peux appliquer dès lundi

Yoan Thirion · green-craft.fr

CC BY SA

Qui suis-je ?

Yoan Thirion

Responsable de la pédagogie, école Coda Dijon

Software Crafter, Coach Agile, juste un dév

GitHub : @ythirion

LinkedIn : yoanthirion



Pourquoi ce sujet

Depuis une dizaine d'années, une question me suit

7 / 9
limites planétaires dépassées

Érosion de la biodiversité
L'intégrité de la biosphère, déjà au-delà du seuil

> Tu crois que tu vas éteindre l'incendie avec quelques gouttes d'eau ?
> Je le sais bien, mais je fais ma part.
>
> La légende du colibri

Comment agir à mon échelle ?
Je viens du numérique : autant commencer par là.

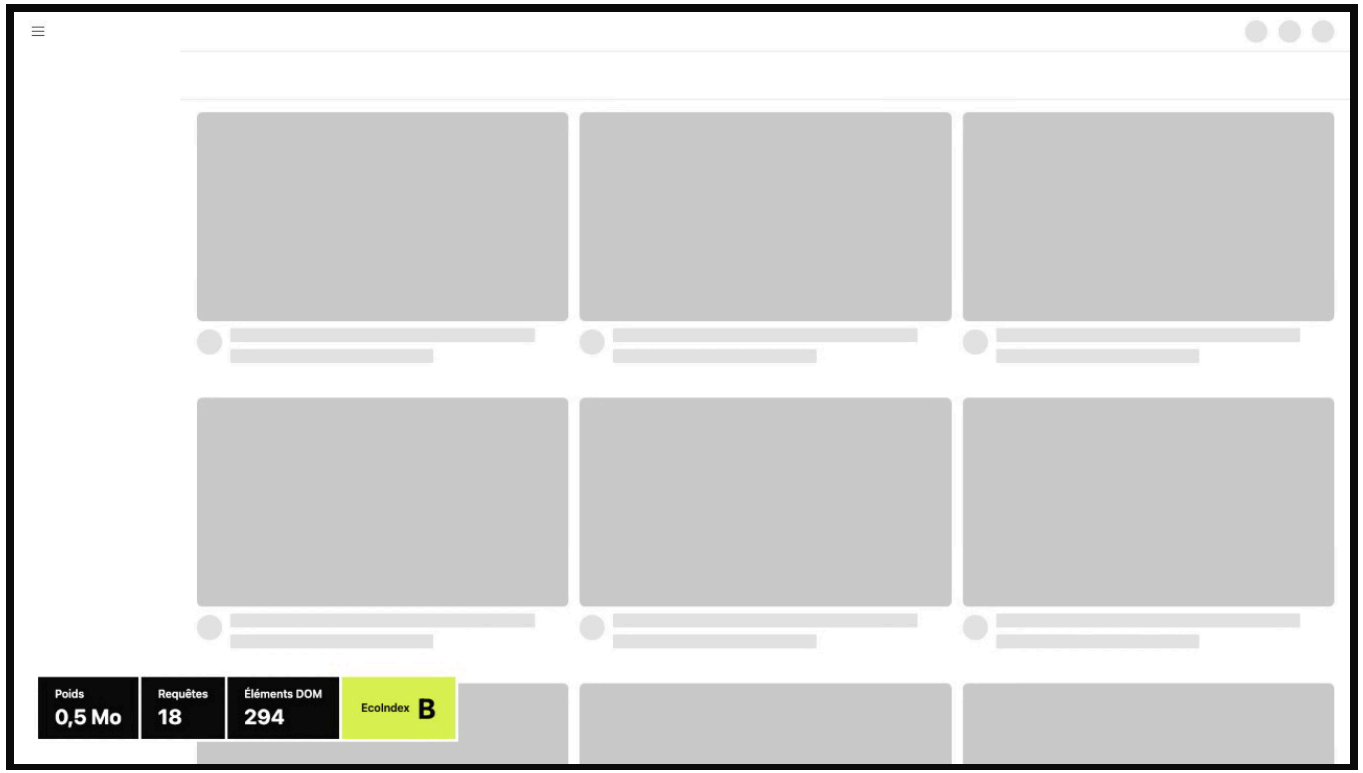


Une vie sur notre planète

Source : Planetary Health Check 2025, Stockholm Resilience Centre et PIK (septembre 2025).

YouTube.com / mesuré le 26 octobre 2026

Un site que j’utilise tous les jours



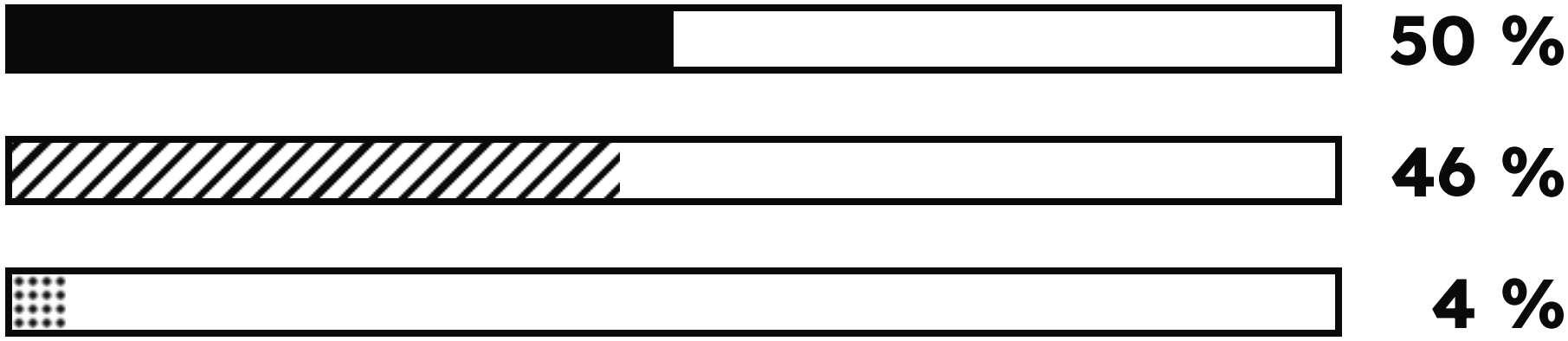
A	B	C	D	E	F	G

Poids	**3,9 Mo**
Requêtes	**125**
Éléments DOM	**1 403**
EcoIndex	**20 / 100**

Une page d'accueil qui n'affiche aucune vidéo.

Source : formule EcoIndex (cnumr), écran 1920 × 1080, sans compte, cookies refusés. Script : scripts/record-ecoindex.mjs.

Où pèse l'empreinte du numérique ?



Notre code fait tourner les serveurs **et** vieillir les terminaux : en tant que devs, nous avons la main sur 96 %.

Source : ADEME-Arcep, mise à jour janvier 2025 (données 2022), empreinte carbone du numérique en France.

Et ce n’est pas que du carbone

Le numérique pèse déjà lourd

40 %
du budget climat annuel soutenable d'un·e internaute pour rester sous +1,5 °C

6
équipements numériques actifs par personne, en moyenne

#1
indicateur environnemental : l'épuisement des **ressources** (minéraux et métaux), devant le climat

Source : GreenIT.fr, « Le numérique, c'est 40 % du budget climat soutenable » (2025).

D’où je parle

Je viens du dev.

Et plus exactement du monde du Craft.

Qualité | Pragmatisme | Responsabilité | Orientation Produit | Feedback

Manifesto for Software Craftsmanship / 2009

Du Craft…

01 Pas seulement des logiciels opérationnels, **mais aussi des logiciels bien conçus**	02 Pas seulement l'adaptation aux changements, **mais aussi l'ajout constant de valeur**
03 Pas seulement les individus et leurs interactions, **mais aussi une communauté de professionnels**	04 Pas seulement la collaboration avec les clients, **mais aussi des partenariats productifs**

Source : manifesto.softwarecraftsmanship.org, qui prolonge le Manifeste Agile (2001).

Et si les valeurs du Craft pouvaient orienter le logiciel vers un numérique plus responsable ?

… au Green Craft

01 Des partenariats productifs permettant de co-construire des solutions **conscientes de leur impact environnemental et sociétal**	02 Des logiciels bien conçus, ET aussi **sobres et durables**
03 Pas seulement ajouter constamment de la valeur, mais aussi **de la valeur respectueuse des ressources**	04 Plus qu'une communauté de professionnels, une communauté dédiée à **un numérique soutenable**

Passer de l'intention à l'action

Des référentiels en or

RGESN | GR491 / ISIT | 115 bonnes pratiques GreenIT | RGAA | WCAG 2.2 | AFNOR Spec 2201 | ISO/IEC 21031 (SCI) | Référentiel INR

Un quotidien déjà chargé

Cybersécurité | RGPD | Réglementaire | Analytics | Observabilité | Infra / Cloud | FinOps | CI/CD | Dette technique | IA générative

RGESN 1.1 · Prioritaire

« Le service numérique a-t-il été évalué favorablement en termes d'utilité en tenant compte de ses impacts environnementaux ? »

Pas assez concret pour un·e dev

Lundi matin, devant mon IDE : par quoi je commence ?

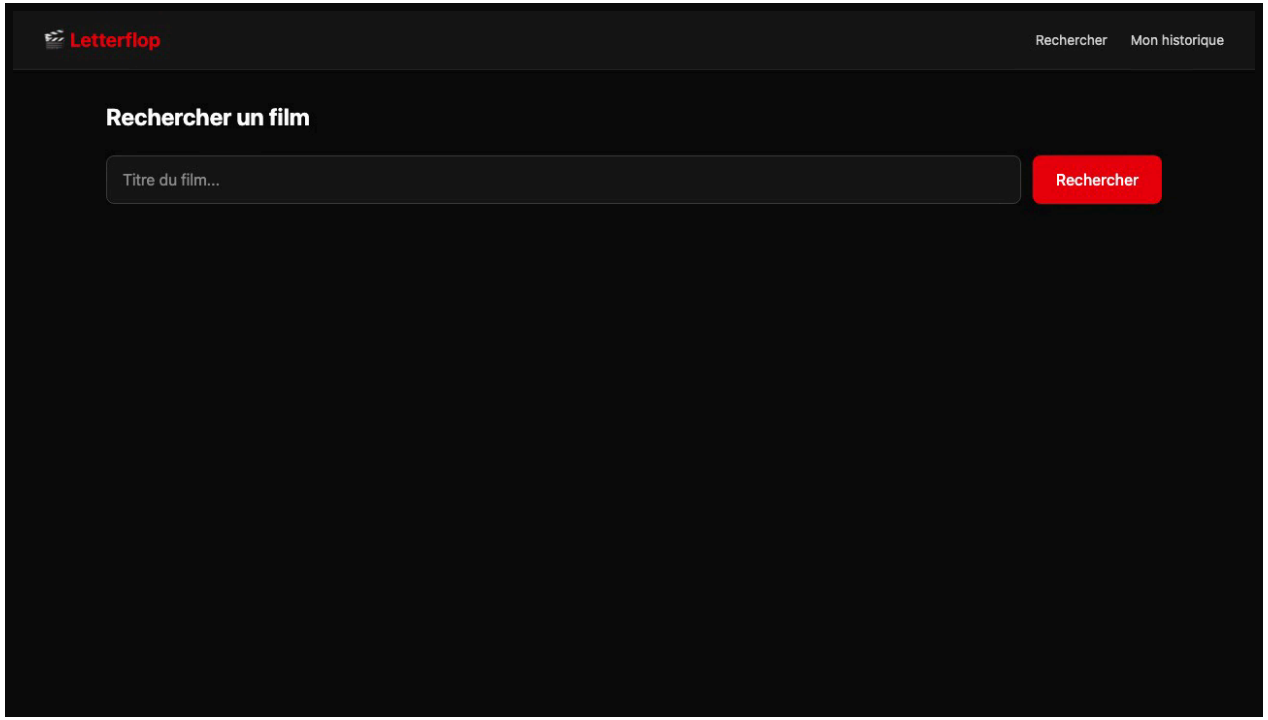
Le fil rouge / étude de cas

Letterflop

Un clone de Letterboxd, le journal de films aux 30 millions de membres.

01 Rechercher un film
02 Consulter sa fiche
03 Le noter, le logger
04 Retrouver son historique

`Spring Boot / Java 25` `TypeScript / Vite` `PostgreSQL` `Docker Compose` `API TMDB`



Une violation par famille du RGESN. Des chiffres avant / après mesurés.

Source : github.com/ythirion/letter-flop (branches main et rgesn). Letterboxd : 30,7 M de membres fin juin 2026, d'après Tiny.

01

Valeur 01 / avant de coder

Des partenariats productifs permettant de co-construire des solutions conscientes de leur impact environnemental et sociétal

Ça veut dire quoi, un partenariat productif ?

Pas seulement la collaboration avec les clients, mais aussi des partenariats productifs.

We are not factory workers

On ne prend pas des commandes : on co-construit la solution.

Say no, for our client's good

Dire non, ou plutôt « pas comme ça », quand c'est son intérêt.

Challenging is our responsibility

Questionner le besoin fait partie du métier, pas de l'insolence.

Pas des code monkeys. Pas des pisseurs de code.

Source : Sandro Mancuso, *The Software Craftsman* (2014). Robert C. Martin, *The Clean Coder* (2011), chapitre « Saying No ».

Comment co-construire des solutions ?

Event Storming

« Un format d'atelier flexible pour explorer ensemble des domaines métier complexes. »

- Tout le monde autour du même mur : métier, PO, devs, UX, ops
- On raconte l'histoire du produit avec des **événements**, au passé
- On fait émerger les **problèmes** et les **opportunités** avant de coder

Big Picture | Process Modelling | Software Design

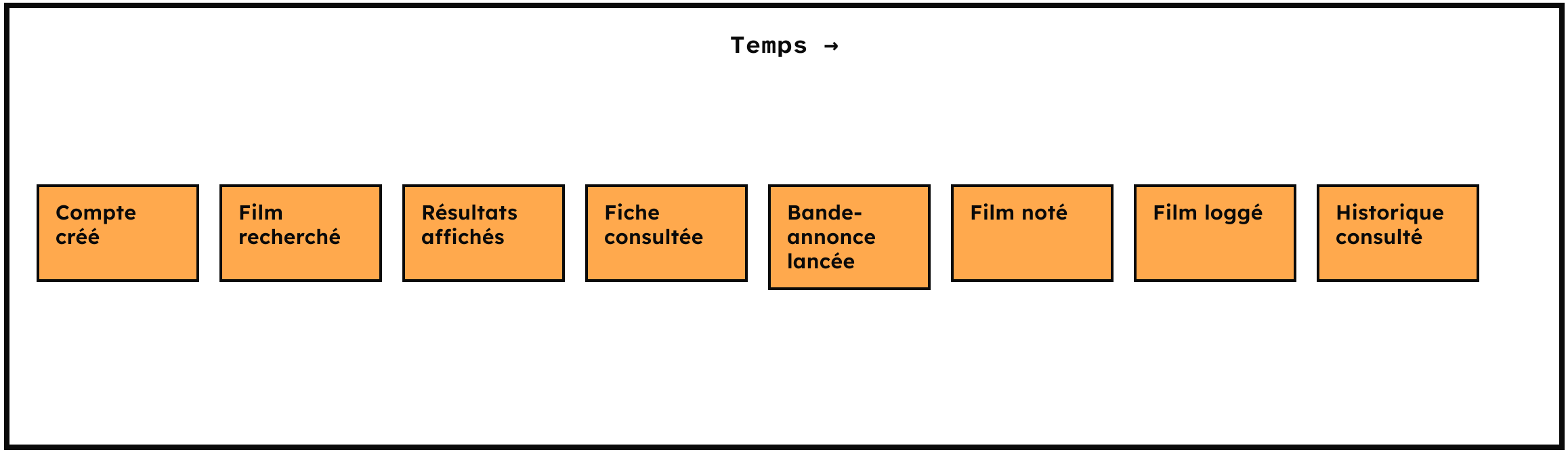


Événement métier **Film noté**	Commande **Noter le film**
Acteur **Cinéphile**	Règle **Dès qu'un film est noté…**
Read model **Note moyenne**	Système externe **TMDB**
Hotspot **Et si TMDB ne répond pas ?**	

Source : eventstorming.com. Notation d'Alberto Brandolini, *Introducing EventStorming*. Photo : Avanscoperta.

Étape 1 / chaotic exploration, puis enforce the timeline

D’abord, raconter le métier dans le temps

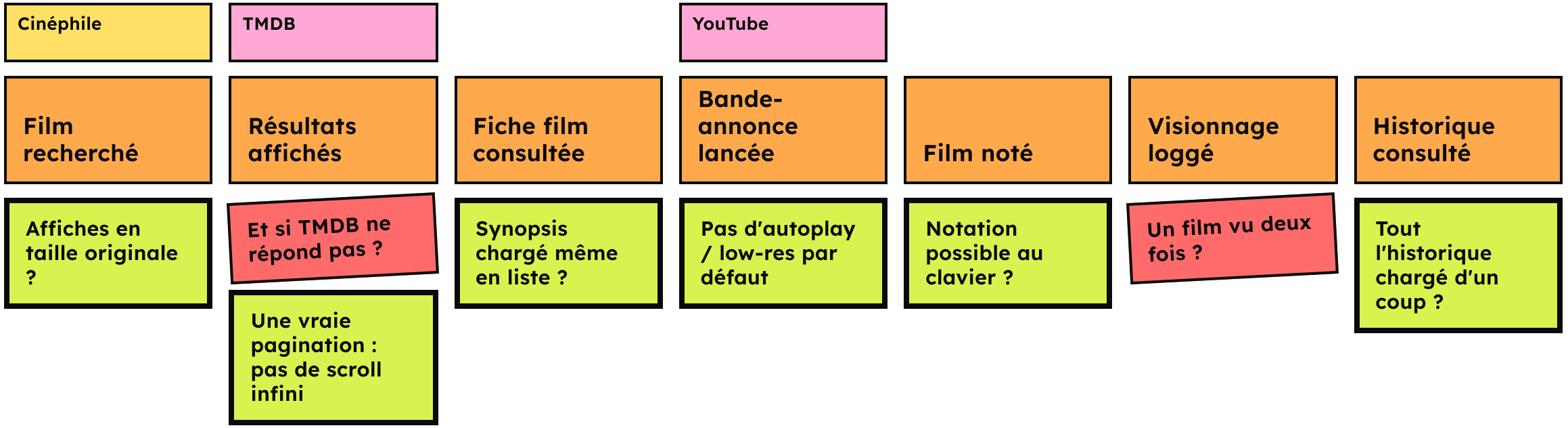


Des événements métier, au passé, puis remis dans l'ordre ensemble.

Source : d'après les étapes « chaotic exploration » et « enforce the timeline » du Big Picture d'Alberto Brandolini.

Événement · Acteur · Système externe · Hotspot · Numérique responsable

Le parcours d'un·e cinéphile



Nouvelle couleur : numérique responsable. Gaspillage, sobriété, accessibilité.
Pas une case à cocher en fin de mur : une couleur centrale, posée sous chaque événement.

Comment challenger un·e PO ?

Une affiche de film générée par IA



La user story

En tant que cinéphile, je veux générer une affiche personnalisée pour chaque film loggé, créée par IA à partir du synopsis et de ma note, afin de garder un souvenir unique de chaque visionnage.

01 **Mini ACV** pour quantifier, avec des hypothèses d'usage

Hypothèse	Basse	Haute
Utilisateur·rices par jour	1 000	1 000
Part qui génère une affiche	10 %	30 %
CO2e par affiche	1,6 g	5 g
Par an	**≈ 58 kg**	**≈ 547 kg**

02 **Prioriser valeur / impact** environnemental et social

03 **Décider** : construire, retravailler ou abandonner

Valeur ↑

Retravailler	**Construire**
Abandonner US-A	**Différer**

← Impact fort · faible → environnemental et social

Source : Luccioni et al., « Power Hungry Processing », FAccT 2024 (Stable Diffusion XL, inférence seule).

Les critères du RGESN comme grille d’arbitrage

Famille « Spécifications » : 5 questions posées à chaque US, avant le sprint.

Critère	Question	US-A Affiche générée par IA	US-B Bande-annonce HD autoplay	US-C Formulaire de log
2.3	Fonctionne sur connexion lente ?	Non	Non	Oui
2.6	Impact évalué avant construction ?	Non	Non	Oui
2.7	Décommissionnable proprement ?	Difficile	Oui	Oui
2.9	Composants UI à l’impact maîtrisé ?	Non	Non	Oui
2.10	Impact des services tiers évalué ?	Non	Moyen	Oui
Décision		**Abandonner**	**Retravailler**	**Construire**

Source : critères de la famille Spécifications du RGESN (Référentiel général d'écoconception de services numériques).

US-B, retravaillée

La grille l'a classée « Retravailler » : on réécrit l'US jusqu'à ce que les critères passent.

Avant

En tant qu'utilisateur·rice, je veux voir la bande-annonce **en HD** directement sur la fiche, **en lecture automatique** à l'ouverture de la page, afin de découvrir le film avant de le logger.

Après

En tant qu'utilisateur·rice, je veux pouvoir **cliquer sur « Voir la bande-annonce »**, en qualité standard par défaut, avec la possibilité de passer en HD, afin de découvrir le film **sans charger une vidéo que je ne regarderai peut-être pas**.

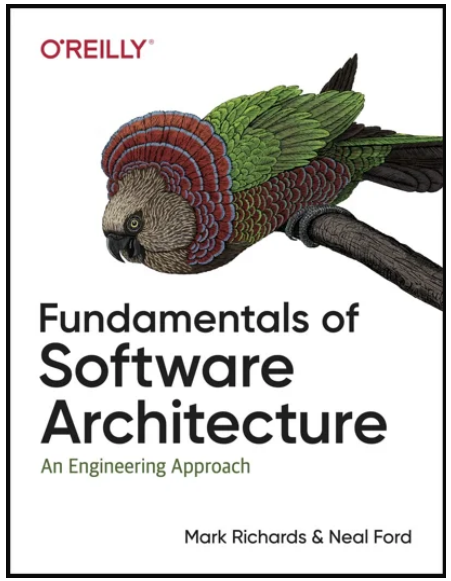
`2.3 · Pas d'autoplay` `2.9 · Vignette au lieu du lecteur` `2.10 · Iframe YouTube chargée au clic` `2.3 · HD au choix`

02

Valeur 02 / concevoir et coder

Des logiciels bien conçus, et aussi sobres et durables

L'architecture, c'est aussi des caractéristiques



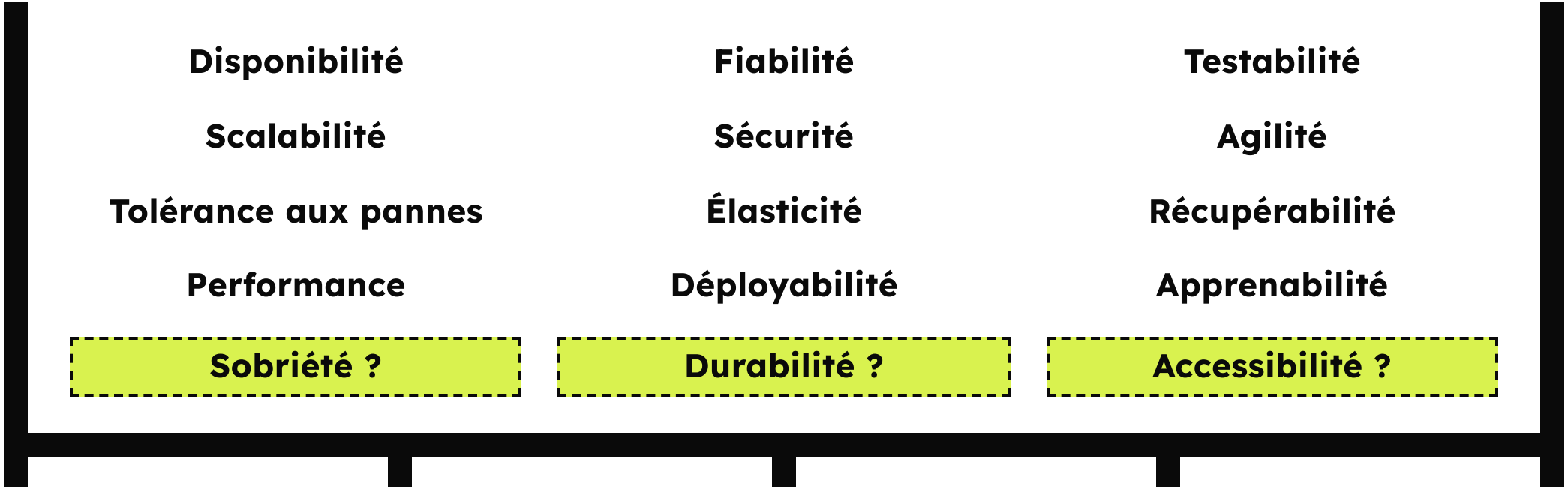
Une caractéristique d'architecture, ou « -ility »

Un critère de succès du système, **orthogonal au fonctionnel**. Elle doit remplir 3 conditions :

1. spécifier une considération de conception **non fonctionnelle**
2. **influencer la structure** du design
3. être **critique** pour le succès de l'application

Source : Mark Richards, Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020).

Les caractéristiques portent le système



Source : d'après Mark Richards, Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020).

Les rendre explicites, et mesurables

> **Fitness function** : un mécanisme qui fournit une évaluation objective de l'intégrité d'une caractéristique d'architecture. Un test, un seuil, une métrique, qui tourne à chaque commit.

Caractéristique	Ce qu'elle exige de la structure	Sa fitness function
Sobriété	Ressources proportionnelles à l'usage, dépendances minimales, seules les données utiles transférées	Budget de taille d'image, budget de payload par endpoint, seuil EcoIndex
Durabilité	Fonctionne sur des terminaux anciens et des connexions lentes, ne force pas le renouvellement	Parcours clés testés en réseau lent et CPU bridé
Accessibilité	Composants natifs et sémantiques, navigation clavier, contrastes	Zéro violation axe-core sur les parcours critiques

Le RGESN et le GR491 deviennent une source de caractéristiques implicites, comme l'expertise métier.

Source : Neal Ford, Rebecca Parsons, Patrick Kua, *Building Evolutionary Architectures* (O'Reilly, 2017). RGESN, RGAA.

Sobriété : un budget qui casse le build

Ce qu'elle exige

Des ressources proportionnelles à l'usage, des dépendances minimales, seules les données utiles transférées.

Le build casse si…

l'image du frontend dépasse **100 Mo**. Aujourd'hui : 62,4 Mo.

CI : budget de taille d'image Docker

```
SIZE=$(docker image inspect letter-flop-frontend \
  --format '{{.Size}}')
BUDGET=$((100 * 1000 * 1000))   # 100 Mo

if [ "$SIZE" -gt "$BUDGET" ]; then
  echo "Image : $((SIZE / 1000000)) Mo > budget 100 Mo"
  exit 1
fi
```

Source : RGESN 3.1, 3.2, 3.7. Même principe pour un budget de payload par endpoint ou un seuil EcoIndex.

Fitness function / durabilité

Durabilité : tester sur un vieux téléphone

Ce qu'elle exige

Fonctionner sur des terminaux anciens et des connexions lentes, sans forcer le renouvellement du matériel.

Le build casse si…

l'historique ne s'affiche pas en **5 s** en réseau lent, avec un processeur bridé ×4.

Playwright : réseau lent et CPU bridé

```
test('historique utilisable sur un vieux terminal', async ({ page }) => {
  const cdp = await page.context().newCDPSession(page)
  await cdp.send('Network.emulateNetworkConditions', {
    offline: false, latency: 400,
    downloadThroughput: 50_000, uploadThroughput: 50_000,
  })
  await cdp.send('Emulation.setCPUThrottlingRate', { rate: 4 })

  await page.goto('/history.html')
  await expect(page.locator('#logs-list > *').first())
    .toBeVisible({ timeout: 5_000 })
})
```

Source : RGESN 2.3 et famille 4. Émulation réseau et CPU via le Chrome DevTools Protocol.

Fitness function / accessibilité

Accessibilité : zéro violation

Ce qu'elle exige

Des composants natifs et sémantiques, la navigation au clavier, des contrastes suffisants.

Le build casse si…

un critère RGAA vérifiable automatiquement échoue sur un parcours critique.

Playwright + axe-core ≥ 4.11 : tag RGAAv4

```
import AxeBuilder from '@axe-core/playwright'

test('recherche de film conforme RGAA', async ({ page }) => {
  await page.goto('/')
  await page.getByRole('searchbox').fill('Dune')
  await page.keyboard.press('Enter')

  const { violations } = await new AxeBuilder({ page })
    .withTags(['RGAAv4'])
    .analyze()
  expect(violations).toEqual([])
})
```

Source : RGAA 4.1. Tags RGAA dans axe-core depuis la version 4.11. Seule une partie des critères est automatisable : ce test ne remplace pas un audit.

Et sur Letterflop, ça donne quoi ?

N'embarquer que l'utile



"You wanted a banana but what you got was a gorilla holding the banana and the entire jungle."

-- Joe Armstrong

Avant — **2,63 Go**

Après — **791 Mo**

Ce qu'on a fait : un multi-stage build

```
FROM maven:3.9-eclipse-temurin-25-alpine AS build
RUN mvn package -DskipTests

FROM eclipse-temurin:25-jre-alpine
COPY --from=build /app/target/*.jar app.jar
```

backend –78 %
JDK et Maven restent dans l'étape de build

frontend –93 %
Vite dev → fichiers statiques servis par nginx

Ce n'est pas une optimisation : c'est retirer ce qui n'aurait jamais dû être là.

Source : Docker, multi-stage builds. Mesures `docker images`, base postgres:15 inchangée (445 Mo). RGESN 3.1, 3.3.

Ne transférer que l'utile

Avant : toute la table, deux fois, paginée en Java

```
List<MovieLog> allLogs =
    repository.findAllOrderByWatchedAtDesc();
List<MovieLog> freshLogs =
    repository.findAllOrderByWatchedAtDesc();
// tri et subList en mémoire…
long total = repository.findAll().size();
```

Ce qu'on a fait : pagination SQL et index

```
return repository
    .findAllByOrderByWatchedAtDescCreatedAtDesc(
        PageRequest.of(page, size))
    .map(this::toDto);
```

```
CREATE INDEX idx_movie_logs_watched_at
  ON movie_logs (watched_at DESC, created_at DESC);
```

+77 %
de requêtes par seconde (79 → 140 req/s)

−45 %
de latence p50 (122 → 67 ms)

−94 %
de données avec `size=20` au lieu de 1 000 (123 → 7,4 Ko)

Source : mesures `ab` sur `GET /api/logs`, branches `main` et `rgesn`, ~330 lignes en base. RGESN 6.5, 7.1.

Un Result plutôt qu'un throw

Avant : une note invalide lève une exception

```
if (rating < 1 || rating > 5)
    throw new InvalidRatingException(rating);
```

Après : une valeur de retour

```
sealed interface Result permits Ok, Err {}

if (rating < 1 || rating > 5)
    return new Err("La note doit être entre 1 et 5");
```

Par appel, pile de	10 appels	100 appels
`throw` + `fillInStackTrace`	2 076 ns	10 922 ns
`throw` sans stack trace	551 ns	5 403 ns
`Result`	10 ns	110 ns

La nuance : sans stack trace, c'est moins cher, mais on perd de quoi diagnostiquer.

Des exceptions pour l'exceptionnel, des valeurs de retour pour l'attendu.

⚠ **Micro-benchmark :** l'écart dépend de votre gestion des erreurs (profondeur de pile, fréquence, try/catch global). Avoid exceptions, Xtrem TDD

Source : Benchmark JMH 1.37, JDK 25, Intel Core i5-8279U.

Rester utilisable sur un vieux téléphone

−75 %

Lazy loading

D'images au premier affichage : seules les affiches visibles sont chargées.

```
<img loading="lazy" width="185"
height="278">
```

−94 %

Des affiches à la bonne taille

Sur une recherche, ~4,5 Mo → ~274 Ko : on demande à TMDB la taille affichée.

```
- /t/p/original
+ /t/p/w185
```

0

Requête vers Google Fonts

Plus de domaine tiers : le texte s'affiche tout de suite, même en réseau lent.

```
body {
  font-family: system-ui;
}
```

Un site fluide sur un vieux téléphone, c'est un téléphone qu'on garde.

Source : mesures DevTools Network sur Letterflop, recherche « Matrix », branches `main` et `rgesn`. RGESN 2.3, 5.x, 6.x.

Accessible, au clavier compris

Des focus visibles

On voit toujours où l'on se trouve en naviguant au clavier.

```
- class="focus:outline-none"
+ class="focus:outline-2"
```

RGAA 10.7

Des intitulés pour les lecteurs d'écran

Le champ de recherche est étiqueté, chaque affiche a un texte alternatif.

```
<input aria-label="Rechercher un film par titre">
<img alt="Affiche du film ${title}">
```

RGAA 1.1, 11.1

Une notation native

Des boutons radio au lieu d'étoiles en JavaScript : Tab, flèches, lecteur d'écran, **zéro JS**.

```
<fieldset>
  <input type="radio" name="rating">
```

RGAA 7.1, 7.3

Le composant natif est aussi le plus sobre.

Source : RGAA 4.1. Vérifié à chaque commit par la fitness function axe-core (tag RGAAv4).

Les caractéristiques d'architecture

Explicites ou implicites

Explicites : écrites dans les exigences.
Implicites : personne ne les écrit, mais le système échoue sans elles.

Opérationnelles

Disponibilité | Performance | Scalabilité | Élasticité | Fiabilité | **Sobriété ?**

Structurelles

Maintenabilité | Extensibilité | Testabilité | Portabilité | **Durabilité ?**

Transverses

Sécurité | Confidentialité | Légalité | Accessibilité

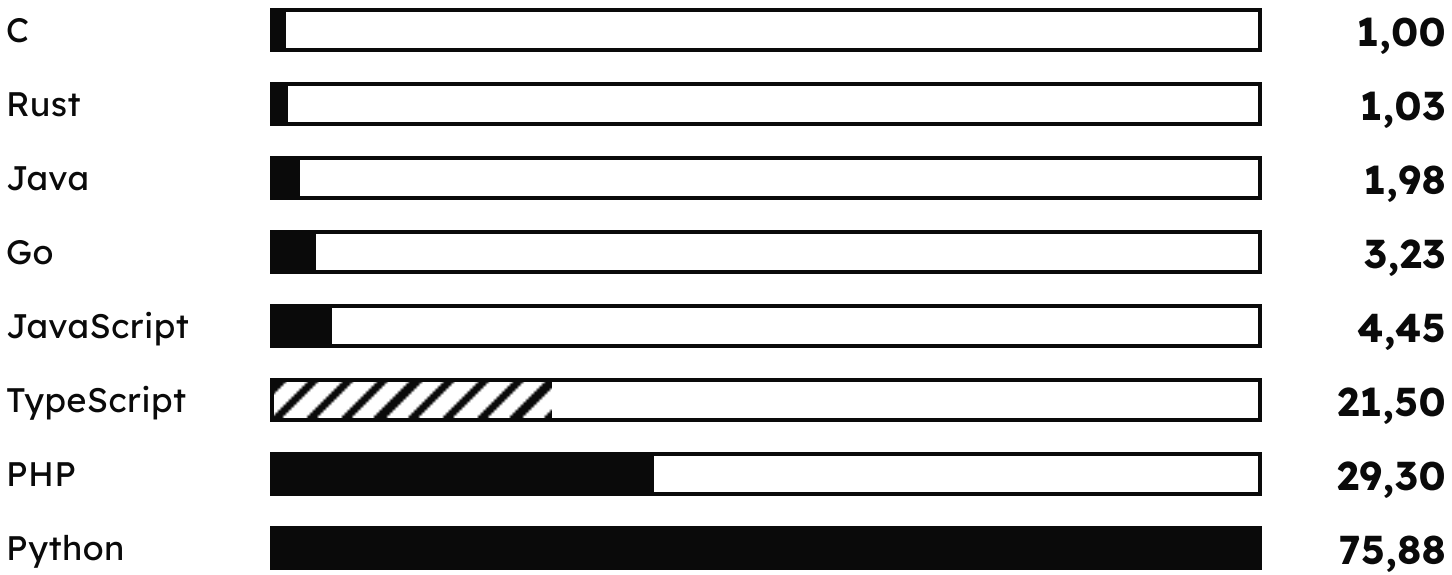
Sobriété absente, accessibilité jamais dans le top 3 : implicites, donc invisibles, donc sacrifiées.

Source : Mark Richards, Neal Ford, *Fundamentals of Software Architecture* (O'Reilly, 2020).

Et nos langages de programmation ?

Au-delà de l’envie, de la hype, de l’orgueil.

Et notre langage, on se pose la question ?



Selon l'usage. Peu d'écart si l'appli attend, beaucoup si elle calcule.

Selon l'utilisation. JavaScript et TypeScript : presque pareils, et pourtant ×5.

Toujours un compromis. Java : sobre en énergie, gourmand en mémoire.

Pas de bon langage : un choix selon l'usage.

Source : Pereira et al., « Energy Efficiency across Programming Languages », SLE 2017. Énergie consommée, C = 1.

Deux trajectoires / un même choix

Rust et Python qui explosent

Rust : sobre, choisi pour les logiciels très utilisés

÷3 — **de ressources** pour le même service chez Cloudflare, qui fait transiter une grande partie du web.

Python : gourmand, et le plus utilisé au monde

6 sur 10 — **développeur·ses l'utilisent**, porté par l'essor de l'IA.

×76 — **d'énergie** par rapport au C, pour faire la même chose.

Plus un logiciel est utilisé, plus le choix du langage compte.

Source : Cloudflare, « How we built Pingora », 2022. Stack Overflow Developer Survey 2025 (57,9 %). Pereira et al., SLE 2017 (×75,88).

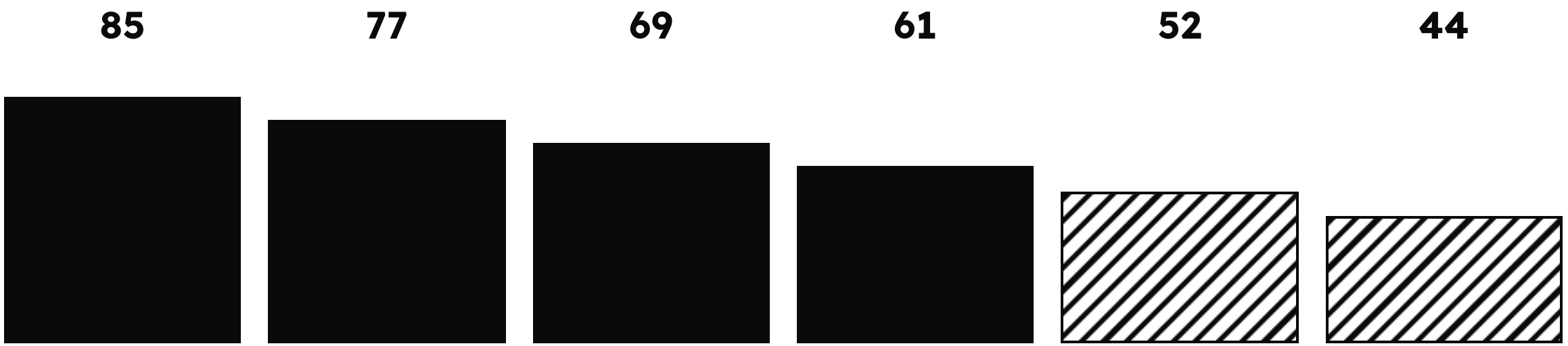
Valeur 03 / mesurer en continu

03

Pas seulement ajouter constamment de la valeur, mais aussi de la valeur respectueuse des ressources

Le constat / sans mesure, rien ne se voit

Sprint après sprint, l'empreinte dérive



Sprint 1
La version optimisée

Sprint 2
+ une bibliothèque d'icônes

Sprint 3
+ un carrousel en page d'accueil

Sprint 4
+ un outil de statistiques

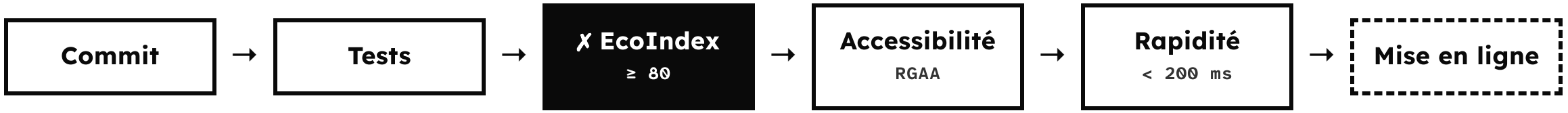
Sprint 5
+ un widget de chat

Sprint 6
+ une vidéo d'accueil

Personne n'a rien cassé. Personne n'a rien vu.

Source : valeurs illustratives. EcoIndex : un score sur 100 calculé à partir du poids de la page, du nombre de requêtes et de la complexité du DOM (ecoindex.fr).

Un contrôle qui bloque, comme un test



✗ EcoIndex 77 < 80 : le sprint 2 ne passe pas, la mise en ligne est bloquée.

EcoIndex : greenit-analysis-action

```yaml
- name: GreenIT Analysis
  uses: ythirion/greenit-analysis-action@0.1.2
  with:
    url-file: greenit-urls.yaml
    min-ecoindex: '80'
```

Rapidité : un benchmark ab

```yaml
- name: API Benchmark
  run: |
    p95=$(ab -n 50 -c 5 "$API/logs?page=0&size=6" \
      | grep "95%" | awk '{print $2}')
    if [ "$p95" -gt 200 ]; then exit 1; fi
```

Source : greenit-analysis-action, basée sur GreenIT-Analysis CLI. Accessibilité et performance : les fitness functions de la valeur 02. RGESN 1.4.

Un rapport, à chaque push

Scénario	EcoIndex	Part due aux actions	Eau (cl)	GES (gCO2e)	Bonnes pratiques à mettre en oeuvre
Historique					
> Chargement de la page	86 A	0	1.92	1.28	3
Fiche film					
> Chargement de la page	85 A	0	1.95	1.3	3
Recherche					
> Chargement de la page	85 A	6	1.95	1.3	3

Bonne pratique	Effort	Impact	Priorité	Grade
Ajouter des expires ou cache-control headers (>= 95%)	3	4	4	❌
Compresser les ressources (>= 95%)	9	9	9	❌
Eviter les requêtes en erreur	9	9	9	❌

Le score dit où on en est, les bonnes pratiques disent quoi corriger.

Source : rapport réel GreenIT-Analysis CLI sur Letterflop (branche `rgesn`), 26/10/2026. 24 des 115 bonnes pratiques CNumR sont vérifiées automatiquement.

Mesurer la rapidité, pas la deviner

BENCHMARK DE CHARGE

On envoie des centaines de requêtes, on chronomètre

Requêtes par seconde	**79 → 140**	**+77 %**
Temps de réponse médian	**122 → 67 ms**	**−45 %**

L'historique de Letterflop, avant et après pagination et index.

MICRO-BENCHMARK

On isole un bout de code, on le répète des millions de fois

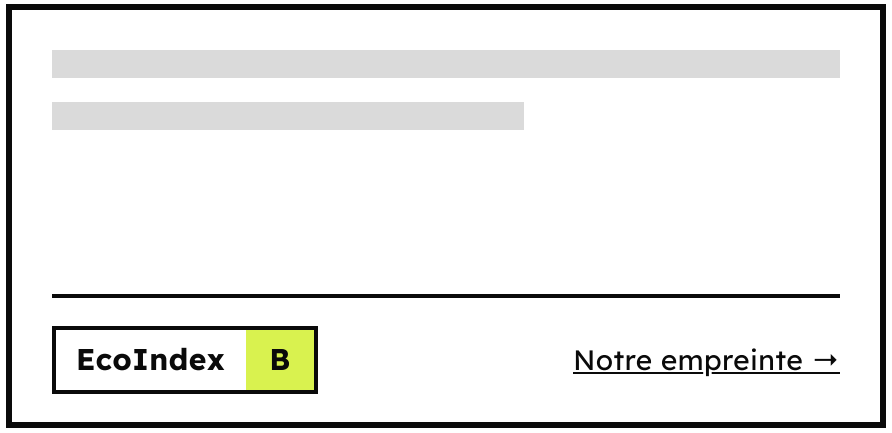
`throw`, pile de 10 appels	**2 076 ns**	
`Result`	**10 ns**	**÷200**

Le même cas métier, deux façons de l'écrire.

Un benchmark montre aussi ce qui se dégrade : −10 % sur la fiche film.

Source : ApacheBench 2.3 (100 requêtes, 10 en parallèle, `GET /api/logs`) et JMH 1.37 (JDK 25, Intel Core i5-8279U). Comparer toujours sur la même machine, avec les mêmes données.

Afficher son score, en pied de page



Le badge officiel du collectif CNumR, mis à jour à chaque mesure, cliquable vers le détail.

~1,3 Ko — **au total** : un badge de 779 octets et un script de 569 octets. Le badge ne pèse presque rien.

7 jours — **de cache** : le score n'est pas recalculé à chaque visite.

```
<div id="ecoindex-badge"></div>
<script src="https://cdn.jsdelivr.net/gh/cnumr/ecoindex_badge@3/assets/js/ecoindex-badge.js" defer></script>
```

On affiche le Nutri-Score sur les aliments. Pourquoi pas sur nos sites ?

Source : cnumr/ecoindex_badge. Note affichée à titre d'exemple. RGESN 1.5.

Une page « Notre empreinte »

ooo `letterflop / empreinte`

Notre empreinte numérique

Ce que nous avons optimisé

- **–80 %** sur le poids de l'application installée
- **–94 %** de données sur l'historique
- **~4,5 Mo → ~274 Ko** d'images par recherche
- **0** police téléchargée

Comment nous mesurons

- EcoIndex : poids, requêtes, DOM
- Accessibilité : critères RGAA automatisables
- Chaque modification est mesurée avant mise en ligne

Nos engagements

- Aucune mise en ligne sous un EcoIndex de 80
- Le seuil ne redescend jamais
- Vérifiez par vous-même : ecoindex.fr

Ce qu'on affiche publiquement, on s'interdit de le dégrader.

Démo : **`green-craft.fr/empreinte`**

Source : chiffres mesurés sur Letterflop, branches `main` et `rgesn`. Atelier « RGESN en action », exercice 14. RGESN 1.5.

04

Valeur 04 / partager

Plus qu’une communauté de professionnels, une communauté dédiée à un numérique soutenable

Partager, y compris ses échecs

Ses mesures

Avant, après, et la méthode. Comme les chiffres de Letterflop dans ce talk : chacun peut les refaire.

Ses pratiques

Un Event Storming responsable, une grille d'arbitrage, un seuil en CI : ce qui marche chez nous peut marcher ailleurs.

Ses échecs

La fiche film qui perd 10 %, l'affiche IA abandonnée. Ce qui n'a pas marché fait gagner du temps aux autres.

Ses outils

En open source : la GitHub Action GreenIT, Letterflop, l'atelier « RGESN en action ».

On a tous à y gagner : on progresse plus vite ensemble.

La règle des 2 : choisis 2 pratiques, applique-les cette semaine.

Et partage le résultat.

Ta checklist pour lundi

01 Des partenariats conscients de leur impact

- [ ] Event Storming , avec une couleur numérique responsable
- [ ] Challenger son PO : pas « non », mais « pas comme ça »
- [ ] Critères RGESN « Spécifications » comme grille d'arbitrage
- [ ] Mini ACV pour quantifier une feature
- [ ] Matrice valeur / impact environnemental et social

02 Des logiciels sobres et durables

- [ ] Sobriété, durabilité, accessibilité en caractéristiques
- [ ] Une fitness function par caractéristique
- [ ] Multi-stage build pour les images Docker
- [ ] Pagination SQL et index, pas de `findAll()`
- [ ] `Result` plutôt que `throw` pour l'attendu
- [ ] Lazy loading, images à la bonne taille, polices système
- [ ] Focus visibles, champs étiquetés, composants natifs

03 De la valeur respectueuse des ressources

- [ ] Une quality gate EcoIndex qui bloque la mise en ligne
- [ ] Accessibilité et temps de réponse vérifiés à chaque commit
- [ ] Un seuil ambitieux, qui ne redescend jamais
- [ ] Le badge EcoIndex en pied de page
- [ ] Une page « Notre empreinte »

04 Une communauté soutenable

- [ ] Partager ses mesures, ses pratiques, ses échecs
- [ ] Ouvrir ses outils en open source
- [ ] Monter une communauté, ou porter le sujet ailleurs
- [ ] Rejouer l'atelier « RGESN en action » en équipe

Pour aller plus loin

L’atelier RGESN en action



Pas une énième prise de conscience : **une boîte à outils** à appliquer dès lundi, sur ton produit, dans tes affinages, dans ton code.

- **15 exercices** pratiques, de la mesure au bilan
- **Letterflop** en fil rouge, branche `main` à corriger
- Les **9 familles du RGESN** couvertes
- **1 journée** en mob ou pair programming, sans prérequis

github.com/ythirion/rgesn-en-action

Bien conçus et sobres, ça ne se décrète pas : ça se pratique.

Merci ! Des questions ?

ythirion.github.io/journee-econception2027/green-craft-journee-econception-2027.pdf